Dynamic Financial Product Discovery Platform

Course	Web Programming
Instructor	Arsalan Khan
Submitted by	Muhammad Ahsan / 23i-5010

1. Introduction

Project Overview

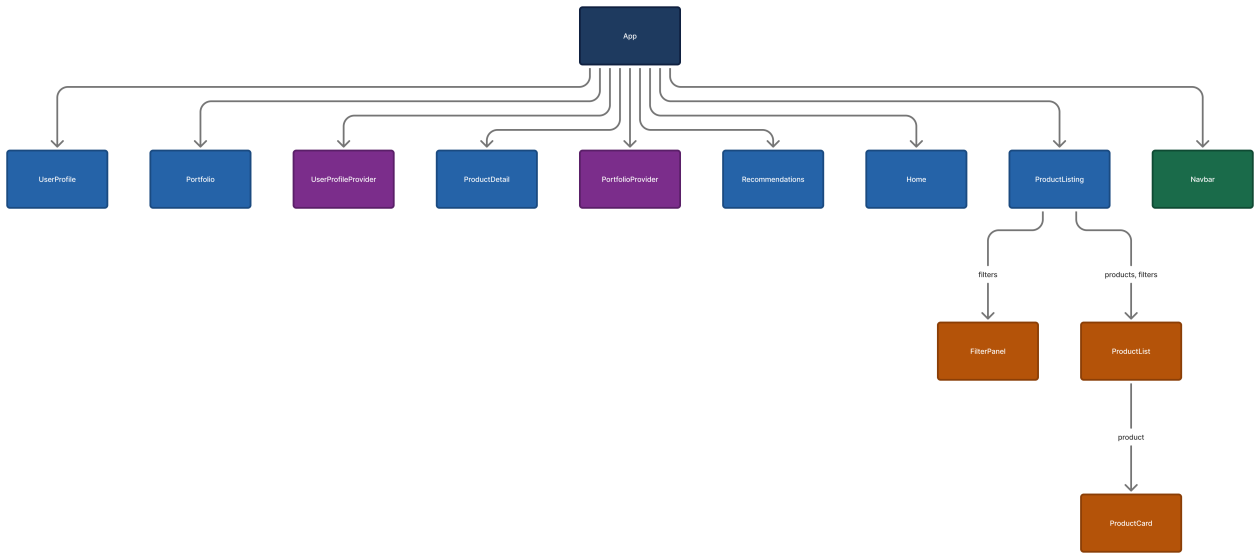
FinVest is a React-based web application that lets users browse financial products, build an investor profile, and get product recommendations that actually match their situation. The idea behind it is simple: most investment platforms assume the user already knows what they want. This one does not. It takes a person's risk tolerance, budget, and time horizon and does the matching for them. All the product data comes from an external API and gets transformed into proper financial instruments before it is ever shown on screen.

Features Implemented

Feature	Description
Dynamic Product Catalog	30 products fetched from a live API, transformed into financial instruments, and displayed in a filterable grid
Multi-Criteria Filtering	Filter by risk level, return range, category, liquidity, time horizon, and investment cap, all at once
Investor Profile Builder	A five-field form that captures risk tolerance, horizon, budget, liquidity preference, and investment goal
Recommendation Engine	Matches the saved profile against the full catalog and returns a sorted shortlist the user can act on
Portfolio Management	Add products, adjust allocations, and watch total invested, weighted return, and risk distribution update live
Consistent Data	A seeded mathematical function ensures every product shows the same financial data on every page load

2. Component Architecture

Component Hierarchy Diagram



Component Descriptions and Props Flow

`App` sits at the top of the tree and does the heavy lifting for data fetching. It keeps the `products` array, a `loading` flag, and an `error` message in local state, and passes those three values as props to any page that needs them. It also wraps everything in `PortfolioProvider` and `UserProfileProvider` so those contexts reach every component without any prop threading.

`Products` (the product listing page) receives `products`, `loading`, and `error` from App. It owns its own filter state and uses it to derive a visible product list on every render. That list gets split out into individual `ProductCard` components, while `FilterPanel` sits alongside and drives filter changes through a callback.

`FilterPanel` is stateless. It receives the current filter object and an `onFilterChange` callback from its parent. Every time a user touches a filter control, it fires that callback and lets the parent re-derive the filtered list.

`ProductCard` receives one `product` object as a prop, displays its key details, and connects to `PortfolioContext` through the `usePortfolio` hook to check whether that product is already in the user's stash. The "Add to Stash" button calls `addToPortfolio` directly on the context, with no props involved.

`Portfolio` receives no props at all. It pulls everything it needs straight from `PortfolioContext`: the list of stashed items, the live stats object, and the functions to update or remove allocations. This keeps it completely independent of the component above it.

---

3. State Management

Local State Usage

`useState` handles UI state that is scoped to a single component and does not need to be visible anywhere else.

The clearest example is the profile form in the `Profile` page. Each of the five fields (risk tolerance, investment horizon, monthly capacity, liquidity preference, and investment goal) is a controlled input tied to its own piece of local state. The form stays in sync with whatever the user types because the input reads from state and writes back to it on every keystroke.

The `Products` page stores all active filter values in a single local state object. When the user changes any filter through `FilterPanel`, the object updates and the component re-renders immediately with a freshly derived filtered list. There is no secondary state and no effect involved.

In `App`, `loading` and `error` are two simple values that wrap the fetch call. `loading` goes true when the request starts, then back to false once it finishes. `error` gets a message if the request fails so the pages receiving it know to show a fallback view instead of an empty grid.

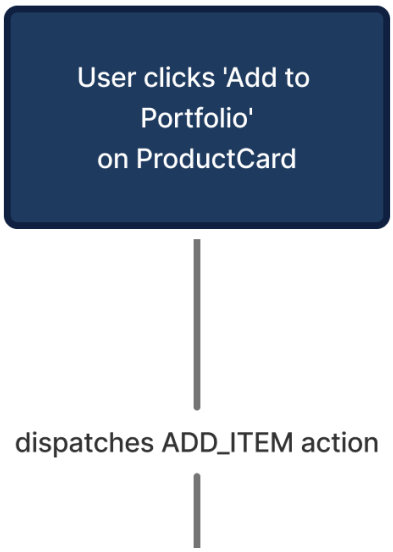
Context API Implementation

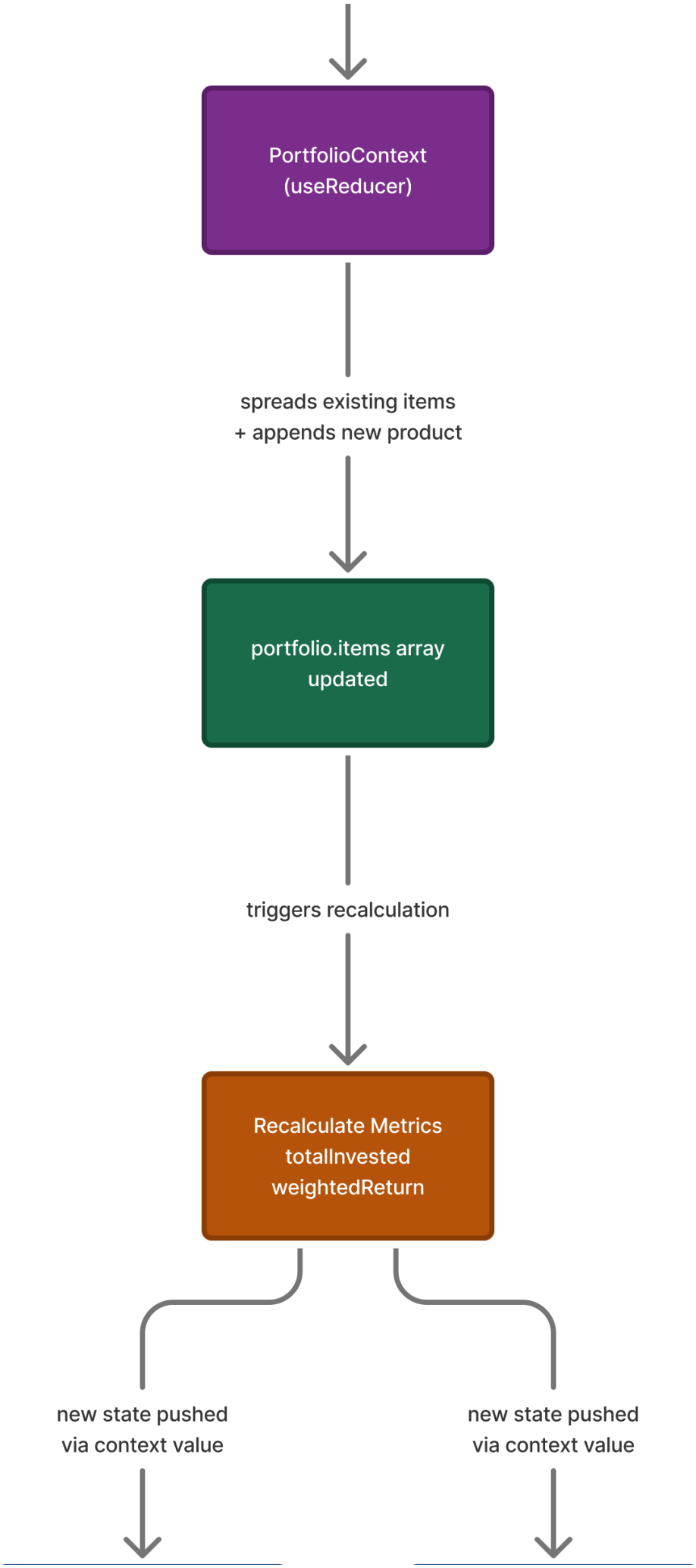
Two contexts handle state that multiple unrelated components need to access. Without them, the same data would have to be passed as props through several layers of components that do not use it themselves.

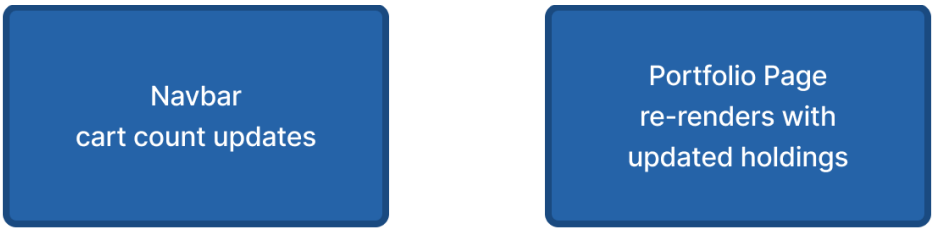
`PortfolioContext` manages the user's stash. It holds an `items` array where each entry pairs a `product` with an `allocatedAmount`. Four functions are exposed: `addToPortfolio` (with a duplicate guard), `removeFromPortfolio`, `updateAllocation`, and `isInPortfolio`. A `stats` object computed via `useMemo` sits alongside these, recalculated on every change so any component reading total invested, weighted return, or risk distribution always gets the current number.

`UserProfileContext` holds the saved investor profile as a plain object. `updateProfile` replaces it when the form is submitted. `isProfileComplete` checks that all five required fields are filled in. `getProductRecommendations` calls the recommendation utility and returns a filtered, sorted list based on the current profile. The Recommendations page uses this directly, which means the profile never has to travel down the component tree as a prop.

State Flow Diagram




PortfolioContext
(useReducer)
spreads existing items
+ appends new product
portfolio.items array
updated
triggers recalculation
Recalculate Metrics
totalInvested
weightedReturn
new state pushed
via context value
new state pushed
via context value



---

4. Financial Logic

Data Model Explanation

Every product that comes out of the transformation pipeline carries six financial fields on top of its display data.

Field	Description
`category`	One of four sectors: savings, investment, insurance, or crypto
`riskLevel`	Low, medium, or high, set by the category
`expectedReturn`	A percentage per annum, within a band set by the risk level
`liquidity`	Easy, moderate, or locked, based on how accessible the capital is
`timeHorizon`	Short, medium, or long, matched to the risk level
`minInvestment`	Entry cost in PKR, derived from the product's original retail price

All six fields are guaranteed to be present on every product, which is what makes the filtering and calculation code clean.

Risk and Return Mapping

The expected return bands are fixed and match realistic investment logic:

Risk Level	Return Range	Categories Mapped
Low	3% to 7% p.a.	Savings, Insurance
Medium	7% to 12% p.a.	Investment
High	12% to 27% p.a.	Crypto

Within each band, the exact return for a product is not random. A seeded sine-based function takes the product's numeric ID as input and produces a stable decimal between 0 and 1. That decimal is scaled into the appropriate range. The same product ID always produces the same return, so the data is consistent on every render without any persistence layer.

Recommendation Algorithm Flowchart

Recommendation Algorithm Explanation

The engine takes the full product list and the saved profile. It runs four sequential filters, then sorts whatever remains.

Step 1: Budget filter. Any product with a `minInvestment` higher than the user's monthly capacity is cut. There is no point recommending something the user cannot afford.

Step 2: Risk filter. The tolerance map works cumulatively. Conservative allows only low-risk. Moderate allows low and medium. Aggressive allows all three. Products outside the allowed set are dropped.

Step 3: Time horizon filter. Same cumulative logic. A short horizon sees only short-term products. A long horizon sees all three timeframes. This stops a user on a one-year plan from being sent a five-year product.

Step 4: Liquidity filter. Easy preference means only liquid products. Locked preference means everything passes. Again cumulative, so greater flexibility never shrinks the list.

Sort. Conservative profiles see results sorted by return ascending (lowest risk first). All other profiles see them sorted descending (highest return first).

Portfolio Calculations

All three metrics are computed in `portfolioCalc.js` and refreshed via `useMemo` every time the `items` array changes.

Total Invested is a plain sum of every `allocatedAmount` in the portfolio.

Weighted Return accounts for the fact that not every product has the same allocation. For each product, its allocation share (`allocatedAmount / totalInvested`) is multiplied by its `expectedReturn`. Those values are summed to produce a single blended return percentage, rounded to two decimal places.

Risk Distribution groups the allocated amounts by risk tier (low, medium, high), then expresses each group as a percentage of the total invested. This tells the user at a glance how concentrated their portfolio is in any one risk category.

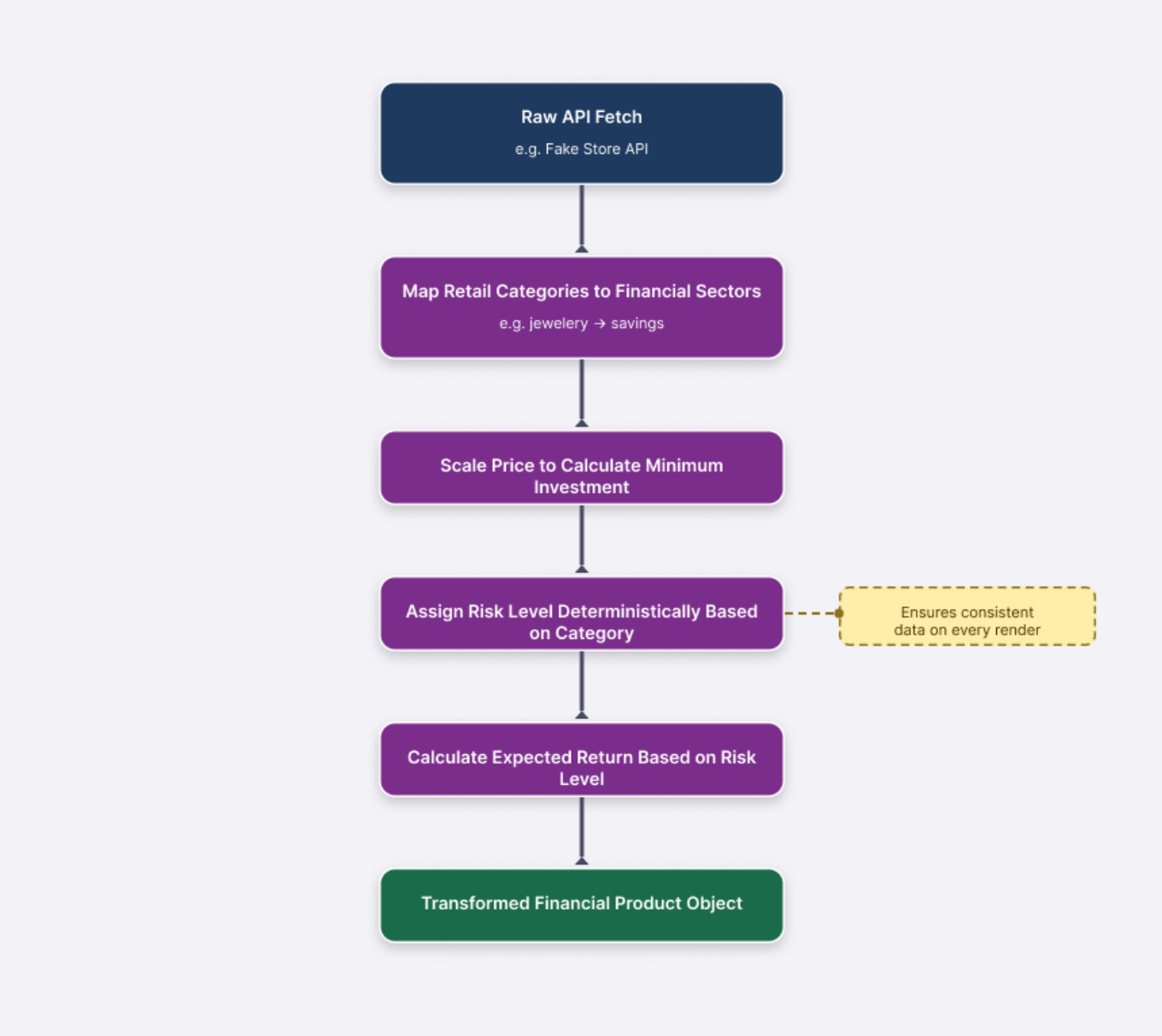
---

5. API Integration

API Choice

The application uses an external mock API (DummyJSON) to fetch retail product data, which is then transformed into financial instruments before anything is shown to the user.

API Data Transformation Pipeline



Transformation Logic

DummyJSON returns ordinary retail products: a title, price, category, description, and thumbnail. The `transformToFinancialProduct` function in `transformData.js` turns each one into a financial instrument through a fixed, deterministic pipeline.

The retail category is thrown out. The product's numeric ID is used instead: `id % 4` assigns it to one of savings, investment, insurance, or crypto. Since the ID is fixed, the category is always the same.

That category is fed through three lookup maps to get the risk level (`RISK_MAP`), liquidity (`LIQUIDITY_MAP`), and time horizon (`TIME_HORIZON_MAP`).

The expected return comes from a seeded sine function that turns the product ID into a stable decimal, scaled into the correct return band for that risk level.

The minimum investment is simply `Math.round(price * 1000)`. A product at $15.99 becomes PKR 15,990. The scaling is arbitrary but consistent, and it produces figures that feel realistic for the Pakistani market context.

Error Handling

The fetch runs inside an async function called `fetchProducts`, which lives in a `useEffect` in `App.jsx`. A `try/catch` block wraps everything from the network request to the JSON parse. If either step fails, the `catch` block writes a user-readable message to the `error` state. Pages that receive `error` as a prop show a fallback view when it is set, rather than leaving the user looking at an empty list. A `cancelled` flag in the `useEffect` cleanup prevents any state update from firing if the component unmounts before the request finishes.

---

6. Challenges and Solutions

Technical Challenge: Multi-Criteria Filtering and Render Loops

The first version of the filter system put the filtering logic inside a `useEffect` that watched the filter state. When a filter changed, the effect ran, computed the new visible list, and wrote it to a separate piece of state. That state update triggered a re-render, which re-evaluated the effect's dependencies, which triggered the effect again. In some cases it would settle; in others it produced stale data depending on the timing of user interactions.

The fix was to stop treating the filtered list as state. It is derived data: it can always be computed from the raw filter values and the product array. Pulling the computation out of `useEffect` and running it inline during the render cycle removed the feedback loop entirely. The `Products` component now stores only the raw filter values. The visible list is calculated fresh on every render from those values. No effect, no intermediate state, no synchronization issue.

Logic Challenge: Stale Weighted Return After Allocation Changes

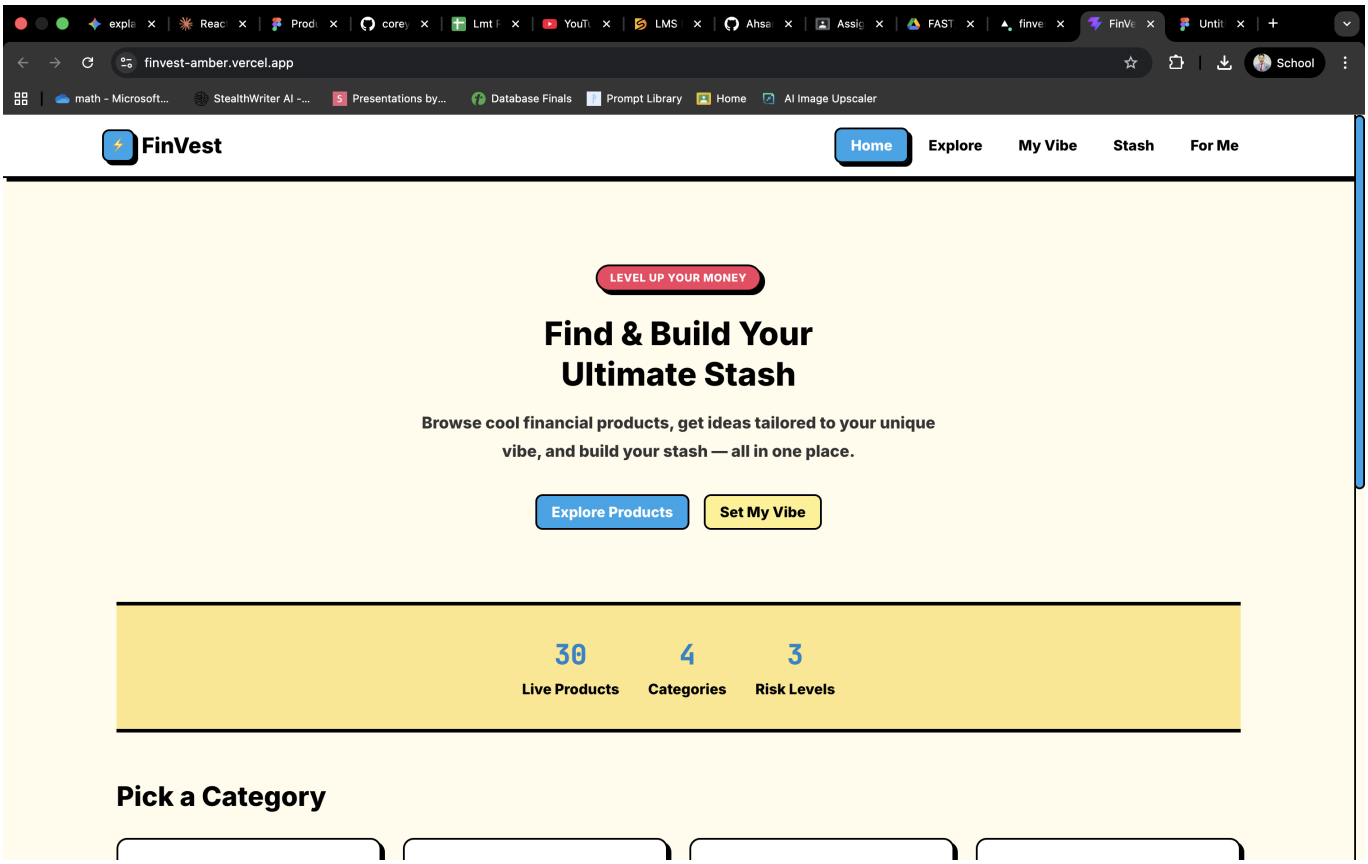
A quieter bug appeared in the portfolio calculations. The stats object was originally computed inside the `Portfolio` page component. When a user changed an allocation amount, the local state updated correctly and the component re-rendered, but the stats were calculated from a stale closure over the old `items` array. The weighted return always showed the value from the previous interaction, never the current one.

Moving the calculation into `PortfolioContext` under `useMemo` solved it. The context's `items` array is the single source of truth. When `items` changes, `useMemo` recomputes `stats` synchronously in the same update cycle. Every component reading from the context sees the correct figures immediately, with no extra wiring required.

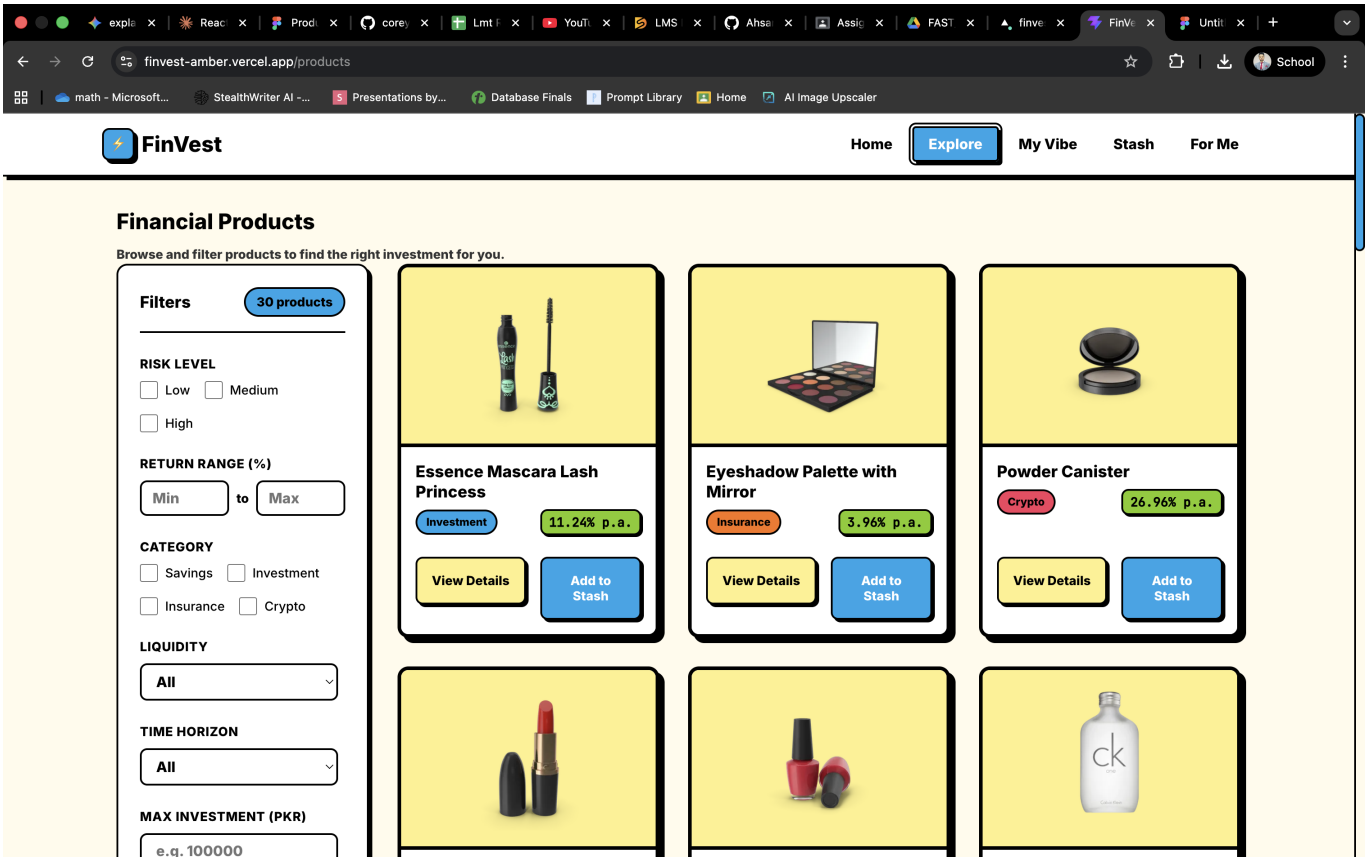
---

7. Screenshots

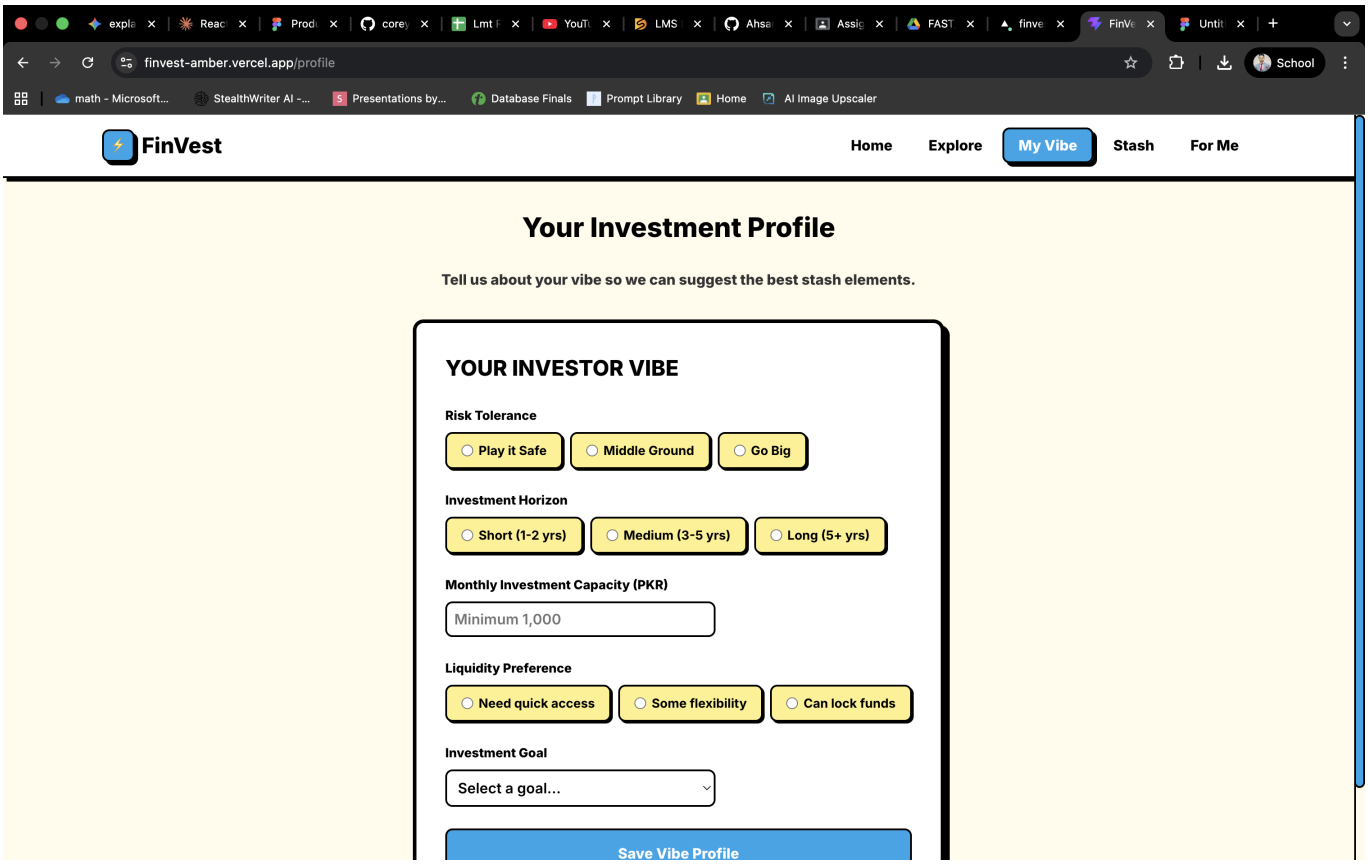
Home Page

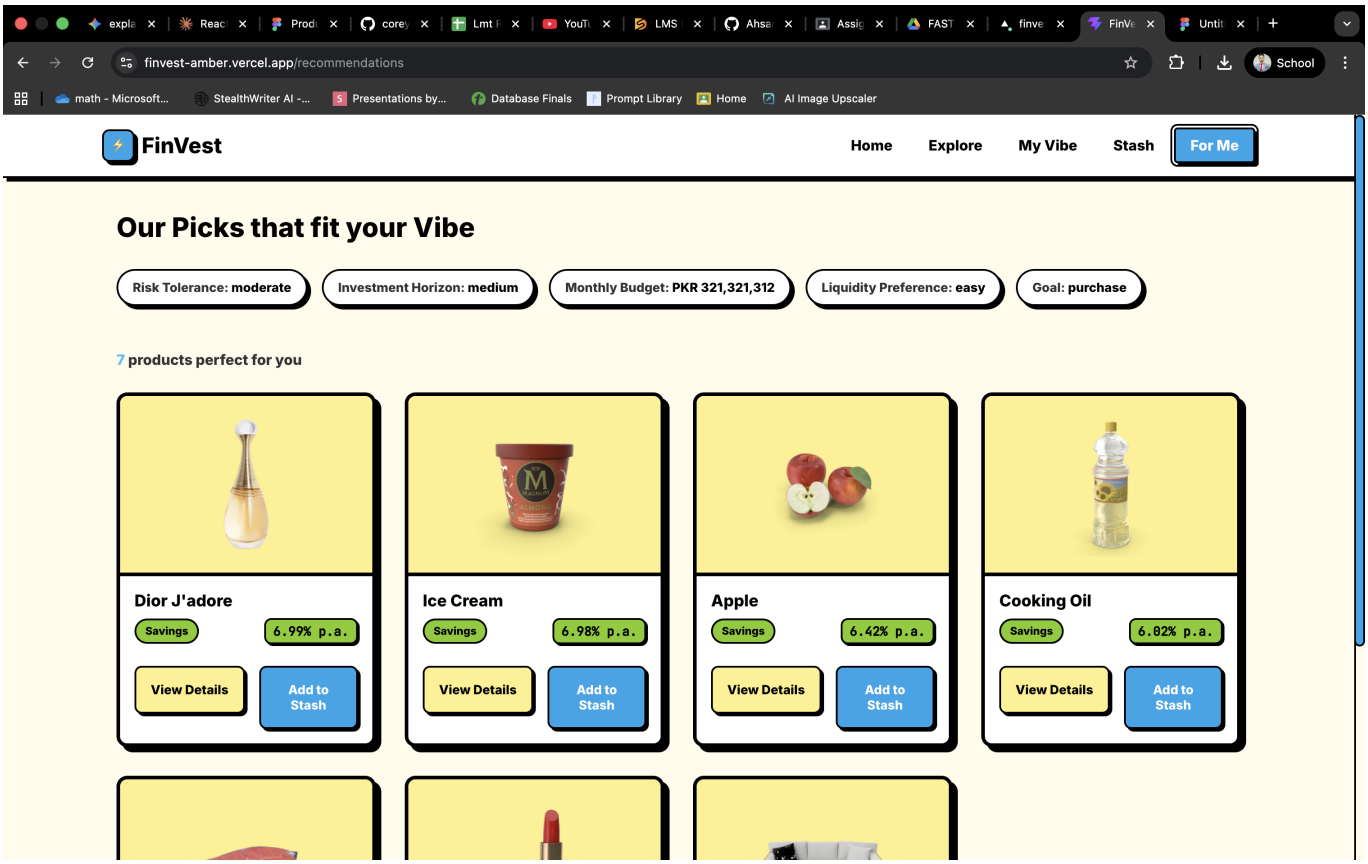


Product Listing Page

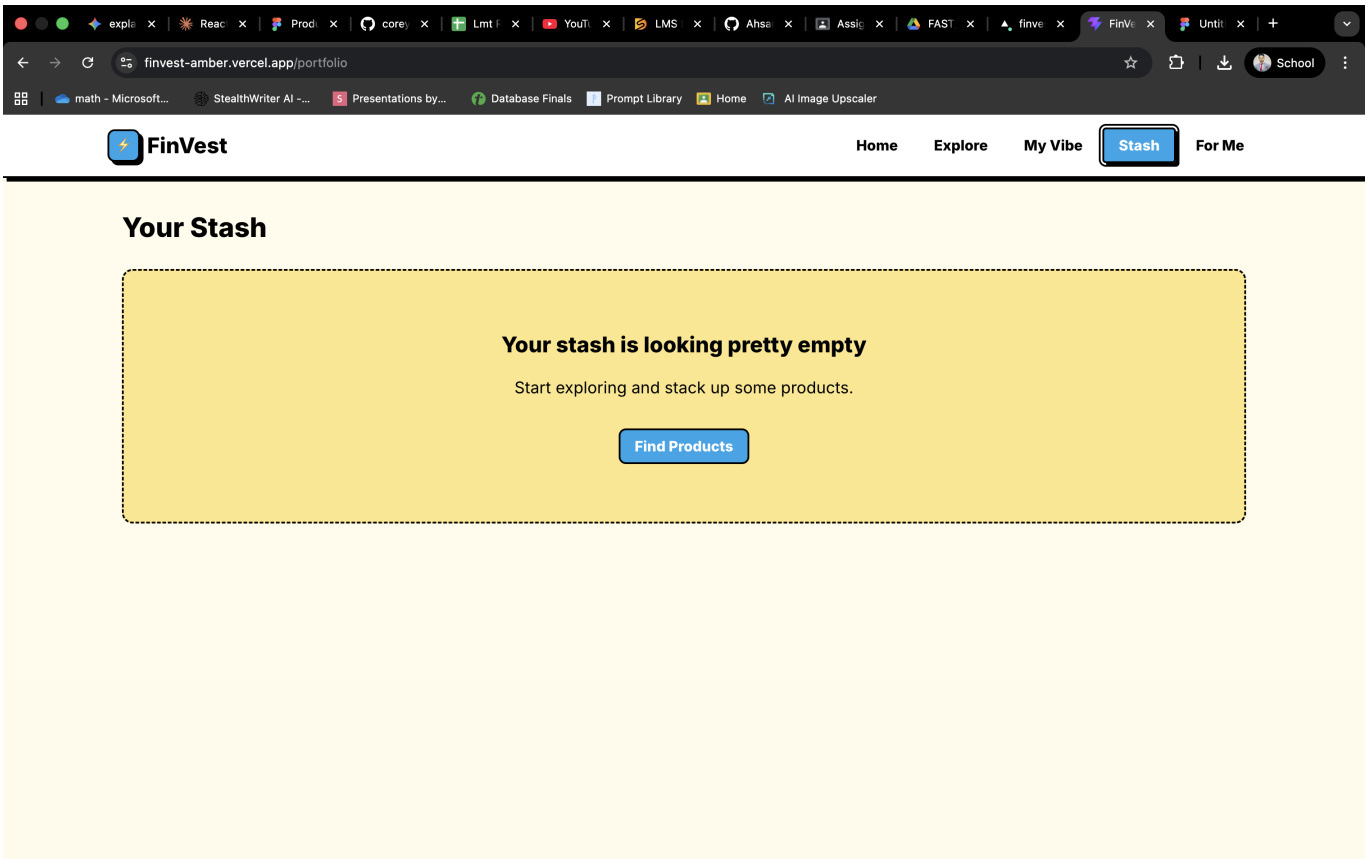


User Profile and Recommendations





Portfolio Page



---

8. Conclusion

What Was Learned

The hardest part of building FinVest was not the UI. It was keeping state consistent when multiple components care about the same data. The two Context providers addressed that, but only after learning that derived data should never be stored in its own state variable when it can just be computed during render. The deterministic transformation functions were another lesson: building them properly upfront meant the data was predictable across every test run, which made debugging far simpler than it would have been with random or mutable data.

Future Improvements

Persistent storage. Right now the portfolio and profile vanish on every page refresh. Connecting to a backend, whether that is Firebase or a simple Node.js API, would let users keep their data across sessions. That is a basic requirement for any application that expects users to return.

Data visualizations. The risk distribution numbers are currently plain text. A pie chart for the risk breakdown and a projected returns graph over the selected time horizon would make the portfolio page significantly easier to read and would bring the experience closer to what a real investment dashboard looks like.